

# Execution-Verified AI Feedback for Code Generation: An Implementation and Negative Result

Anonymous

## Abstract

Code generation models can be improved through post-training feedback, but existing methods face a trade-off: execution feedback (test pass/fail) provides ground-truth correctness but no semantic guidance, while AI feedback (LLM critique) offers rich explanations but is wrong 61% of the time. We propose Execution-Verified AI Feedback (EVAF), which filters AI-generated code critiques through unit test execution before using them as training signals. By accepting only AI suggestions that pass tests, EVAF aims to combine execution’s fidelity with AI’s semantic richness. We implement EVAF using CodeT5-770M and CodeLlama-7b-Instruct on HumanEval (164 problems). However, our experimental validation failed: the pipeline crashed at 53% completion due to infrastructure issues (likely GPU memory exhaustion), preventing measurement of the target accept rate metric. The hypothesis remains untested rather than refuted. We release our implementation and report this negative result to inform future work requiring infrastructure hardening for combined large model inference and dynamic code execution.

## 1 Introduction

Large language models can explain code errors in rich detail, describing why a variable is misnamed, why an algorithm handles edge cases incorrectly, or why a data structure choice leads to inefficiency. Yet these explanations are wrong more than half the time. Analysis of Self-Refine shows that 94% of its failures trace to bad feedback: 33% identify the wrong location and 61% propose incorrect fixes [Madaan et al., 2023]. Meanwhile, execution-based feedback from unit tests provides ground-truth correctness signals but offers no insight into *why* code fails—only *that* it fails.

This tension between **semantic richness** and **signal fidelity** has driven two parallel lines of research. Execution feedback methods like CodeRL [Le et al., 2022] and RLTF [Liu et al., 2023] use test pass/fail signals as reinforcement learning rewards, achieving strong results on code generation benchmarks. AI feedback methods like Self-Refine use LLM-generated critique for iterative refinement at test time. These approaches have been developed and evaluated in isolation, with no controlled comparison on the same model and benchmark setup.

We observe that fidelity and richness are **orthogonal dimensions**. High fidelity does not preclude high richness; the two properties simply have not been combined. This observation motivates a simple question: what if we use execution to *verify* AI feedback rather than replace it?

We propose **Execution-Verified AI Feedback (EVAF)**, a mechanism that filters AI-generated code critiques through unit test execution before using them as training signals. The intuition is straightforward: AI suggestions that break tests are rejected; suggestions that maintain or improve correctness are accepted. The surviving feedback should retain AI’s semantic explanations while inheriting execution’s correctness guarantee.

EVAF operates as follows. Given a code generation problem where a baseline model produces incorrect output, an AI feedback model generates a critique and proposed fix. This fix is applied tentatively and executed against the problem’s unit tests. If all tests pass, the critique is accepted as verified feedback; otherwise, it is rejected. The accept rate—the fraction of AI suggestions surviving verification—determines whether EVAF is viable: too low (<10%) and EVAF degenerates to pure execution feedback; too high (>90%) and the gating adds no value.

We implement EVAF using CodeT5-770M as the baseline generator and CodeLlama-7b-Instruct as the feedback model, targeting HumanEval’s 164 problems. Our implementation comprises seven modules: data

loading, baseline model wrapper, feedback model wrapper, execution gating with sandboxed test execution, metrics computation, visualization, and pipeline orchestration.

**Our contribution is primarily architectural and negative.** We report that the EVAF mechanism is implementable using existing tools, but our experimental validation failed: the pipeline crashed at 53% completion (87/164 problems) before any metrics could be computed. The hypothesis remains untested rather than refuted—infrastructure failure, not theoretical refutation. We release our implementation and analysis to enable future work with proper infrastructure hardening.

The paper proceeds as follows. Section 2 positions EVAF within execution and AI feedback literature. Section 3 describes the EVAF mechanism and implementation. Section 4 details our experimental setup. Section 5 reports the incomplete results. Section 6 discusses implications and limitations. Section 7 concludes with concrete next steps.

## 2 Related Work

### 2.1 Execution-Based Feedback for Code Generation

Execution feedback uses unit test outcomes as training signals. **CodeRL** [Le et al., 2022] pioneered actor-critic reinforcement learning for code, using test pass/fail as episode-level rewards. A critic network predicts functional correctness, enabling critical sampling for code regeneration. CodeRL achieves 2.69 pass@1 on APPS with critic sampling.

**RLTF** [Liu et al., 2023] extends execution feedback with multi-granularity signals. Beyond binary pass/fail, RLTF extracts fine-grained error locations from unit test output, providing line-level localization. Online RL with real-time data generation yields state-of-the-art results: 1.45 pass@1 on APPS without critic sampling. RLTF demonstrates that granularity matters—fine-grained feedback outperforms coarse rewards.

**PPOCoder** [Shojaee et al., 2023] combines PPO-based RL with non-differentiable execution feedback and structure alignment. The framework is task-agnostic, achieving strong results across APPS and MBPP benchmarks.

These methods share a limitation: execution feedback provides no semantic guidance about *why* code fails.

### 2.2 AI-Generated Feedback for Code

AI feedback uses LLM-generated critique for code improvement. **Self-Refine** [Madaan et al., 2023] demonstrates zero-shot iterative refinement: the same LLM generates code, critiques it, and refines based on its own feedback. No training is required—refinement occurs at inference time. Self-Refine achieves +8.2% improvement on code optimization tasks.

However, AI feedback suffers from low fidelity. Madaan et al. analyze Self-Refine failures: 33% stem from identifying the wrong error location, and 61% from proposing incorrect fixes.

**RefineCoder** [Zhou et al., 2025] introduces Adaptive Critique Refinement using LLM-as-Judge and LLM-as-Critic for self-generated code refinement during training.

### 2.3 The Gap

No study directly compares execution and AI feedback under controlled conditions. Each method uses different base models, benchmarks, and training protocols. Furthermore, no prior work treats execution as a *verification mechanism* for AI feedback.

## 3 Methodology

### 3.1 Fidelity and Richness: An Orthogonal Framework

We decompose feedback quality into two independent dimensions:

**Signal Fidelity:** The probability that feedback is factually correct. Execution feedback achieves near-perfect fidelity. AI feedback has low fidelity ( $\sim 39\%$  correctness).

**Semantic Richness:** The amount of explanatory content about *why* code is wrong. Execution feedback has minimal richness. AI feedback is maximally rich.

We observe these dimensions are orthogonal. A method can achieve high fidelity *and* high richness if we filter rich signals through a fidelity check.

## 3.2 EVAF: Execution-Verified AI Feedback

EVAF treats execution as a verification layer for AI feedback:

1. **AI Critique Generation:** Given problem description and failing code, generate critique and proposed fix.
2. **Code Extraction:** Parse fenced code blocks from AI response.
3. **Execution Gating:** Execute extracted fix against unit tests (3.0s timeout, subprocess isolation).
4. **Accept/Reject Decision:** Accept if all tests pass; reject otherwise.

## 3.3 Accept Rate as Viability Indicator

- **<10%:** EVAF degenerates to pure execution feedback
- **20–60%:** AI provides substantive verified feedback (viable)
- **>90%:** Execution gating unnecessary

## 3.4 Implementation Architecture

Seven Python modules:

- **config.py:** Fixed configuration constants
- **data.py:** HumanEval loading via HuggingFace
- **model.py:** BaselineModel (CodeT5-770M) + FeedbackModel (CodeLlama-7b-Instruct)
- **gating.py:** Code extraction + sandboxed subprocess test execution
- **metrics.py:** Accept rate, coverage, rejection breakdown
- **visualize.py:** Three required figures
- **train.py:** Pipeline orchestration

# 4 Experimental Setup

## 4.1 Research Question

Can EVAF produce filtered AI feedback with accept rate in the viable 20–60% range?

## 4.2 Dataset

**HumanEval:** 164 Python programming problems with unit tests.

### 4.3 Models

- **Baseline:** CodeT5-770M (Salesforce/codet5-large)
- **Feedback:** CodeLlama-7b-Instruct (codellama/CodeLlama-7b-Instruct-hf)

### 4.4 Evaluation Protocol

1. Generate baseline code for all 164 problems
2. Filter to failing problems
3. Run EVAF gating on each failing problem
4. Compute accept rate and secondary metrics

### 4.5 Success Criteria

**Pass:** Accept rate 20–60%

**Fail:** Accept rate <10% or >90%

## 5 Results

### 5.1 Experiment Outcome: Incomplete

**Status:** FAILED (infrastructure crash at 53% completion)

| Stage               | Status      | Progress      |
|---------------------|-------------|---------------|
| Baseline generation | COMPLETE    | 164/164       |
| Failure filtering   | COMPLETE    | —             |
| EVAF gating         | FAILED      | 87/164 (~53%) |
| Metrics computation | NOT REACHED | —             |

Table 1: Experiment execution status.

**Start:** 2026-08-18T13:59:28Z

**Termination:** 2026-08-18T14:31:19Z (stalled)

### 5.2 Failure Analysis

Hypothesized root causes:

1. **GPU Memory Exhaustion (HIGH):** CodeLlama-7b float16 requires ~14GB
2. **Subprocess Timeout Escalation (MEDIUM):** Cascading timeouts from pathological code
3. **System-Level Kill (MEDIUM):** External OOM killer

### 5.3 Artifacts Generated

All implementation modules complete. Missing: results.json, metrics.json, figures.

### 5.4 Gate Verdict

**MUST\_WORK** gate **NOT satisfied**—no metrics available.

## 6 Discussion

### 6.1 What This Failure Does NOT Tell Us

- That EVAF does not work
- That accept rates are outside viable range
- That the fidelity  $\times$  richness framework is incorrect

The hypothesis remains **untested**, not refuted.

### 6.2 What This Failure DOES Tell Us

**Infrastructure requirements for EVAF:**

1. Sufficient GPU memory ( $>14$ GB for 7B models)
2. Robust error handling for pathological code
3. Checkpointing for long-running experiments

**Computational overhead:**  $\sim 2\times$  inference cost plus subprocess overhead.

### 6.3 Limitations

1. Single model pair tested
2. HumanEval only
3. No training experiments
4. No baseline comparison

### 6.4 Recommended Next Steps

1. Add checkpointing to gating loop
2. Profile GPU memory; consider 4-bit quantization
3. Complete existence experiment
4. If viable, proceed to comparative evaluation

## 7 Conclusion

We set out to answer whether AI feedback could be made reliable through execution verification. The answer remains unknown.

We proposed EVAF, implemented it (7 modules), and attempted validation on HumanEval. The experiment crashed at 53%, leaving no metrics to report.

**What we contribute:** The fidelity  $\times$  richness framework; a complete EVAF implementation; infrastructure requirements for future work.

**What we do not contribute:** Evidence that accept rate is viable; evidence that verified feedback improves learning; comparison to baselines.

The question—can execution verify AI feedback?—is worth answering. We hope this implementation enables future work to do so.

## References

- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 21314–21328, 2022.
- Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. Rltf: Reinforcement learning from unit test feedback. *Transactions on Machine Learning Research*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Parshin Shojaei, Aneesh Jain, Sindhu Tipirneni, and Chandan K. Reddy. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2301.13816*, 2023.
- Chengqing Zhou, Xiaodong Zhang, Dawu Song, et al. Refinecoder: Iterative improving via adaptive critique refinement. *arXiv preprint arXiv:2502.09183*, 2025.